

EJEMPLO

Cadena de empleados.

Como ejemplo de herencia vamos a pensar en la jerarquía de una empresa. Existen varios tipos de empleados, desde los guardias de seguridad, los de servicio al cliente, los de contabilidad, los de desarrollo, los de ventas, etc. Sobre ellos hay también empleados que actúan como jefes, coordinando y organizando los sectores en los que trabajan.

Sobre todos los empleados está el dueño de la empresa o el CEO.

Pensemos entonces en esa jerarquía, un empleado (de cualquier tipo) se encuentra hasta el fondo, sobre él se encuentra el Jefe de ese departamento. Esta jerarquía se repite para todos los departamentos. Sin embargo, todos los jefes deben de responder al CEO de la empresa, es decir él es el Jefe de Jefes.

Con esta información vamos a crear una jerarquía, empezando de abajo hacia arriba. Entonces empezamos con la clase empleado.

La clase empleado, tiene información básica, como nombre, número de empleado y jefe inmediato, todos los empleados van a tener un jefe, este tiene que ser alguien de su mismo departamento. Como vamos a trabajar solamente con 1 empleado y 1 jefe inmediato, entonces tenemos que el que es Jefe, va a tener de jefe al CEO.

Iría de esta forma *Empleado de Seguridad > Jefe de Seguridad > CEO*.

Entonces vamos a definir las clases empezando por el empleado. Se vería de esta forma.

```
public class Empleado{  
  
    public String nombre;  
    public Integer id;  
    public Jefe jefeInmediato;  
  
    public Empleado(){  
    }  
}
```

Con esa clase ya podemos definir la clase de Jefe.

```
public class Jefe{  
  
    public String nombre;  
    public Integer id;  
    public Jefe jefeInmediato;  
  
    public Jefe(){  
    }  
}
```

¿Qué vemos acá? Tienen los mismos atributos y métodos. Eso significa que estas clases son candidatos a una herencia. Si recordamos, esto significa,

una generalización, es decir, vamos a tomar una clase y vamos a derivar una clase más específica de ella.

En este caso la clase base va a ser Empleado y la clase derivada va a ser Jefe. ¿Por qué? porque los Jefes aún son empleados, y tienen responsabilidades de empleado además de sus responsabilidades de Jefe.

Entonces, vamos a colocarles algunos métodos a la clase empleado. El primero va a ser trabajar. Esto significa que todos los empleados trabajan. El siguiente va a ser: reportar, se usará cuando tenemos que reportar algo a nuestro jefe inmediato. Finalmente, ¿qué vamos a reportar? necesitamos una lista de tareas, entonces agreguemos ese atributo y puede que no queramos reportar solo 1 tarea, entonces vamos a agregar también una lista de tareas completadas. La clase empleado se vería así

```
import java.util.ArrayList;

public class Empleado {
    public String nombre;
    public Integer id;
    public Jefe jefeInmediato;
    public ArrayList<String> tareasPendientes;
    public ArrayList<String> tareasCompletadas;

    public Empleado(String nombre, Integer id){
```

```
        this.nombre = nombre;

        this.id = id;

        this.tareasPendientes = new ArrayList<String>();
        this.tareasCompletadas = new ArrayList<String>();
    }

    public void trabajar(){

        this.tareasCompletadas.add(this.tareasPendientes.remove(0));
    }

    public void reportar(){
        jefeInmediato.reportar(tareasCompletadas, this.id);
    }

    public void asignarJefe(Jefe jefe){
        this.jefeInmediato = jefe;
    }

    public void asignarTarea(String tarea){
        this.tareasPendientes.add(tarea);
    }

}
```

Pero ¿qué es ese método de asignar Jefe? Es la forma en la que se decidió que se comportará el empleado, que puede que no siempre tenga el mismo Jefe en un solo departamento. Esa es una decisión de diseño, si no

considera que el Empleado va a ser asignado a otro Jefe, entonces puede asignar el JefeInmediato directamente en el constructor de la clase.

Ya con eso vamos a colocarles un nuevo atributo a los Jefes, que será una lista de empleados, a los que va a dirigir. Entonces necesitaremos un método que sea asignar empleado, en el cual vamos a asignar el empleado al jefe y al mismo tiempo asignar el jefe al empleado.

Ya que tenemos una lista de empleados vamos a crear otro método: asignar trabajo, este funcionará en conjunto con la lista de tareas que creamos con anterioridad, vamos a guardar una tarea en dicha lista para que el empleado pueda trabajar en ella.

```
import java.util.ArrayList;

public class Jefe extends Empleado{

    public ArrayList<Empleado> empleados;

    public Jefe(String nombre, Integer id){
        super(nombre, id);
        this.empleados = new ArrayList<Empleado>();
    }

    public void asignarEmpleado(Empleado empleado){
        this.empleados.add(empleado);
    }
}
```

```
        empleado.asignarJefe(this);  
    }  
    public void asignarTarea(String tarea){  
        empleados.get(0).asignarTarea(tarea);  
    }  
}
```

El problema acá, si recordamos, es que los Jefes van a reportarse al CEO, no a cualquier otro empleado. También recordamos que los CEO son Jefes, entonces vamos a heredar de la clase Jefe. No colocaremos más atributos ni métodos en la clase CEO, solo nos servirá como referencia.

Notemos dos cosas con los métodos de esta clase. La primera, en el método asignarEmpleado. Aquí recibimos un empleado, lo asignamos a nuestra lista y luego nos colocamos como jefe de dicho empleado. El problema es: recordemos la jerarquía que teníamos: Empleado > Jefe > CEO. Entonces el problema es que no podemos asignar Jefes o CEOs como empleados, un Jefe solamente puede tener a su cargo Empleados y de la forma en la que funciona la jerarquía de clases en Java y como tenemos que Jefe hereda de Empleado (después de todo, siguen siendo empleados) El parámetro empleado puede ser de tipo Jefe (gracias al polimorfismo). Lo que hay que hacer es verificar el tipo actual del objeto. Lo podemos hacer de esta forma:

```
public void asignarEmpleado(Empleado empleado){
```

```
        if(empleado.getClass().getSimpleName().equals("CEO")){  
            System.out.println("No se puede asignar CEO como  
empleado");  
        }else{  
            this.empleados.add(empleado);  
            empleado.asignarJefe(this);  
        }  
    }  
}
```

Aquí estamos verificando que si el empleado que queremos asignar es un CEO, entonces no se permite la asignación. Faltaría verificar si el empleado es también un Jefe.

Sin embargo, en lugar de verificar dos condiciones, es más fácil verificar que solamente si la clase actual es Empleado se pueda asignar ¿Cómo hacemos esto? Queda a modo de tarea al lector. *Pista: Ahí ya tenemos como ver el nombre de la clase, y sabemos que equals nos devuelve verdadero si es igual, entonces, lo que hay que hacer es invertir, o negar, el resultado verdadero de equals.*

La segunda cosa que tenemos que ver en los métodos de la clase es en el método asignarTarea, lo que hacemos es simplemente tomar al primer empleado y asignarle una tarea. Esto es simplemente a modo de ejemplo, una implementación más correcta sería algo así

```
public void asignarTarea(String tarea, Empleado empleado){  
    if(empleados.contains(empleado)){  
        empleado.asignarTarea(tarea);  
    }  
}
```

Aquí verificamos si el empleado en cuestión pertenece a nuestro grupo, y si es así, le asignamos la tarea. ¿Qué problema hay? Que si recordamos de la unidad pasada, `contains` nos devuelve verdadero solamente si es el mismo objeto ¿correcto? incluso implementar el método `equals` no cambia ese valor. El problema es que aquí falta implementar otro método para que `contains` funcione. El método [hashCode](#), este método junto con `equals` son necesarios para la verdadera igualdad entre objetos. Lo explicaremos durante el curso, pero también les invitamos a tratar de construir el método para verificar la igualdad entre objetos.

Finalmente vamos a tratar un poco de excepciones, una excepción es cuando surge un comportamiento inesperado, por ejemplo colocar un nombre donde queremos un número.

Las excepciones no se usan con frecuencia, se usan en casos excepcionales en los cuales no se puede seguir con un flujo normal de ejecución usando solamente IFs y LOOPS. En este caso vamos a usar de ejemplo un caso de una tarea invalida. Lo simularemos usando una tarea en blanco.

Vamos a crear una excepción que se llame `ExcepcionTareaVacia` en la clase `Empleado`

```
class ExcepcionTareaVacia extends Exception{  
    public ExcepcionTareaVacia(){  
        super("No se puede asignar una tarea sin texto");  
    }  
}
```

Y la vamos a utilizar cuando estemos asignando tareas así:

```
public void asignarTarea(String tarea){  
    try{  
        if(tarea.equals("")){  
            throw new ExcepcionTareaVacia();  
        }else{  
            this.tareasPendientes.add(tarea);  
        }  
    }catch(ExcepcionTareaVacia etv){  
        etv.printStackTrace();  
    }  
}
```

Eso levantará una excepción. Solamente estamos mostrando el `stacktrace`, que es el lugar donde se llama la función para saber qué punto del programa está lanzando error, pero lo más común es que en el `catch` tengamos algún código que maneje la excepción.